LAPORAN PRAKTIKUM PEKAN 1

STRUKTUR DATA

Abstrak Data Type (ADT)

Jam

Disusun oleh:

Thaariq Salam

2511532022

Dosen Pengampu: Dr. Wahyudi S.T.M.T

Asisten Praktikum: Sherly Sukmadira

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

TAHUN 2026

KATA PENGANTAR

Puji syukur kami panjatkan kehadirat Allah SWT atas rahmat dan karunia-Nya, sehingga Laporan Praktikum mata kuliah Struktur Data dengan topik "Abstrak Data Type (ADT) " ini dapat diselesaikan secara komprehensif. Dokumen ini disusun sebagai pemenuhan kewajiban akademis pada Praktikum Pekan 1

Praktikum ini berfokus pada perancangan struktur data mandiri melalui kelas `Jam_2511532022` serta implementasi logika kontrol dalam kelas `JamDriver_2511532022` dan `JamDriver2_2511532022`. Melalui pembuatan program ini, penyusun mempelajari cara kerja *Object* dan *Method*, serta bagaimana melakukan validasi input yang adaptif menggunakan perulangan dan kondisi *`if-else`*. Penguasaan materi ini merupakan langkah fundamental untuk merancang alur program yang responsif terhadap variasi data masukan, yang merupakan inti dari logika pemrograman.

Penyusun menyadari bahwa keberhasilan penyusunan laporan ini tidak lepas dari bimbingan dan dukungan berbagai pihak. Oleh karena itu, penyusun menyampaikan terima kasih yang sebesar-besarnya kepada:

1. Bapak Dr. Wahyudi S.T.M.T selaku Dosen Pengampu mata kuliah Algoritma dan Pemrograman.
2. Kak Sherly Sukmadira selaku Asisten Praktikum yang telah memberikan arahan teknis selama pelaksanaan praktikum.

Penyusun berharap laporan ini tidak hanya memenuhi aspek administratif, tetapi juga dapat menjadi dokumentasi yang bermanfaat dan media *review* bagi penyusun maupun pembaca lainnya. Kritik dan saran konstruktif sangat diharapkan demi perbaikan di masa mendatang.

Padang, 1 April 2026

Penyusun

DAFTAR ISI

KATA PENGANTAR i
DAFTAR ISI ii
BAB I 1
PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Tujuan 1
1.3 Manfaat 2
BAB II 3
PEMBAHASAN 3
2.1 Kelas Utama Jam_2511532022 3
2.1.1 Kode program 3
2.1.2 Langkah kerja 5
2.1.3 Analisis hasil 5
2.2 Kelas Penguji Manual JamDriver_2511532022 6
2.2.1 Kode program 6
2.2.2 Langkah kerja 6
2.2.3 Analisis hasil 7
2.3 Kelas Penguji Interaktif JamDriver2_2511532022 7
2.3.1 Kode program 7
2.3.2 Langkah kerja 9
2.3.3 Analisis hasil 9
BAB III 11
PENUTUP 11
3.1 Kesimpulan 11

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman, data tidak selalu sesederhana angka atau huruf saja. Seringkali, kita perlu mengolah data yang lebih kompleks, seperti waktu (jam). Di dalam bahasa pemrograman Java, kita dapat membuat sebuah struktur data mandiri yang disebut Abstract Data Type (ADT). ADT memungkinkan kita untuk membungkus data (jam, menit, detik) dan fungsinya (tambah detik, konversi ke detik) ke dalam satu kesatuan yang disebut Class.

Praktikum kali ini menggunakan tiga komponen utama, yaitu `Jam_2511532022` sebagai model mesin jam, serta dua *driver* (`JamDriver` dan `JamDriver2`) untuk menguji logika tersebut. Pemahaman mengenai bagaimana sebuah Objek diciptakan dari sebuah Class, serta bagaimana Method bekerja untuk memanipulasi data di dalamnya, merupakan pondasi penting dalam pemrograman berorientasi objek (OOP). Selain itu, penggunaan logika kontrol seperti `if-else` dan perulangan `while` sangat diperlukan untuk memastikan bahwa data jam yang dimasukkan oleh pengguna selalu valid dan akurat.

1.2 Tujuan

Pelaksanaan praktikum pekan 1 ini punya target utama agar mahasiswa bisa:

1. Mengimplementasikan ADT: Mampu merancang *class* `Jam_2511532022` yang dapat menyimpan dan mengolah data waktu secara mandiri.

2. Memahami Konsep Objek dan Method: Mempelajari cara menciptakan objek jam dan memanggil fungsi-fungsi (method) yang ada di dalamnya.

3. Menerapkan Logika Validasi: Menggunakan konstruksi kondisi (`if-else`) untuk menyaring input jam, menit, dan detik agar sesuai dengan aturan waktu dunia nyata (0-23 untuk jam, 0-59 untuk menit/detik).

4. Otomasi Perhitungan Waktu: Membuat fungsi yang dapat menghitung durasi antar dua waktu dan melakukan konversi waktu ke dalam total detik secara akurat.

1.3 Manfaat

1. Mempertajam kemampuan berpikir logis dalam menyusun alur program yang adaptif terhadap berbagai masukan data.
2. Membiasakan diri menyusun kode yang terstruktur melalui pemisahan antara logika utama (*Class*) dan pengujian (*Driver*).
3. Memberikan dasar yang kuat bagi penyusun untuk membangun aplikasi yang lebih besar di masa depan, di mana pengelolaan data waktu sangat sering dibutuhkan (seperti sistem absensi atau jadwal).
4. Meningkatkan kepercayaan diri dalam menggunakan *tools* pengembangan seperti Eclipse IDE dan sistem pengarsipan kode seperti GitHub.

BAB II

PEMBAHASAN

2.1 Kelas Utama Jam_2511532022

2.1.1 Kode program

```
package pekan1_2511532022;
public class Jam_2511532022 {
   private int hh; // 0..23
   private int mm; // 0..59
   private int ss; // 0..59
   // --- Validator ---
   public static boolean isValid(int h, int m, int s) {
      return (0 <= h && h <= 23) && (0 <= m && m <= 59) && (0 <= s && s <=
59); }
   // --- Konstruktor ---
   public Jam_2511532022(int h, int m, int s) {
      // Sebaiknya tambahkan validasi di sini juga
      this.hh = h;
      this.mm = m;
      this.ss = s; }
   // --- Selektor ---
   public int getHour() { return hh; }
   public int getMinute() { return mm; } // Perbaikan typo: geMinute -> getMinute
   public int getSecond() { return ss; }
   // --- Mutator ---
   public void setHour(int h) { this.hh = h; }
   public void setMinute(int m) { this.mm = m; } // Perbaikan typo: setHMinute -
> setMinute
   public void setSecond(int s) { this.ss = s; }
   // --- Konversi ---
```

```
    public int toSeconds() {
        return hh * 3600 + mm * 60 + ss; }
    public static Jam_2511532022 fromSeconds(int total) {
        // Gunakan floorMod supaya jika total detik negatif (saat pengurangan),
        // jamnya tetap valid (putar balik ke 23:59:xx)
        int secondsInDay = 24 * 3600;
        int positiveTotal = Math.floorMod(total, secondsInDay);
        int h = positiveTotal / 3600;
        int m = (positiveTotal % 3600) / 60; // Perbaikan logika: m harus sisa dari
jam
        int s = positiveTotal % 60;          // Perbaikan logika: s harus sisa dari menit
        return new Jam_2511532022(h, m, s); }
    // --- Relational ---
    public int compareTo(Jam_2511532022 other) {
        return Integer.compare(this.toSeconds(), other.toSeconds()); }
    public boolean equals(Object o) {
        if (!(o instanceof Jam_2511532022)) return false;
        Jam_2511532022 j = (Jam_2511532022) o; // Harus di-cast dulu
        return hh == j.hh && mm == j.mm && ss == j.ss; }
    public int hashCode() {
        return java.util.Objects.hash(hh, mm, ss); }
    // --- Aritmatika ---
    public Jam_2511532022 plus(Jam_2511532022 other) {
        return fromSeconds(this.toSeconds() + other.toSeconds()); }
    public Jam_2511532022 minus(Jam_2511532022 other) {
        return fromSeconds(this.toSeconds() - other.toSeconds()); }
    public Jam_2511532022 nextSecond() {
        return fromSeconds(this.toSeconds() + 1); }
    // Menambahkan parameter 'n' yang tadi kurang di kode kamu
    public Jam_2511532022 nextNSeconds(int n) {
        return fromSeconds(this.toSeconds() + Math.max(0, n)); }
    public Jam_2511532022 prevSecond() {
```

```
        return fromSeconds(this.toSeconds() - 1); }
    public Jam_2511532022 prevNSeconds(int n) {
        return fromSeconds(this.toSeconds() - Math.max(0, n)); }
    // --- Durasi ---
    public static int durasiDetik(Jam_2511532022 jaw, Jam_2511532022 jakh) {
        return jakh.toSeconds() - jaw.toSeconds(); }
    public String toString() {
        return String.format("%02d:%02d:%02d", hh, mm, ss); }
}
```

2.1.2 Langkah kerja

1) Mendefinisikan atribut `hh`, `mm`, dan `ss` sebagai penyimpan data jam, menit, dan detik.
2) Membuat *Constructor* untuk memberikan nilai awal saat objek jam diciptakan.
3) Mengimplementasikan method `isValid` untuk menyaring angka agar tidak ada jam > 23 atau menit/detik > 59.
4) Menyusun logika matematika *`toSeconds`* untuk mengubah format waktu menjadi total detik tunggal agar mudah dihitung.
5) Membuat method *`toString`* untuk mengatur tampilan keluaran agar selalu berformat `00:00:00`.

2.1.3 Analisis hasil

Kelas ini berhasil berfungsi sebagai "mesin" utama. Analisis menunjukkan bahwa pemisahan logika validasi ke dalam method *`isValid`* sangat efektif untuk mencegah error data di kemudian hari. Penggunaan rumus

TotalDetik = (hh×3600) + (mm×60) + ss

terbukti akurat dalam mengonversi waktu ke satuan detik.

2.2 Kelas Penguji Manual JamDriver_2511532022

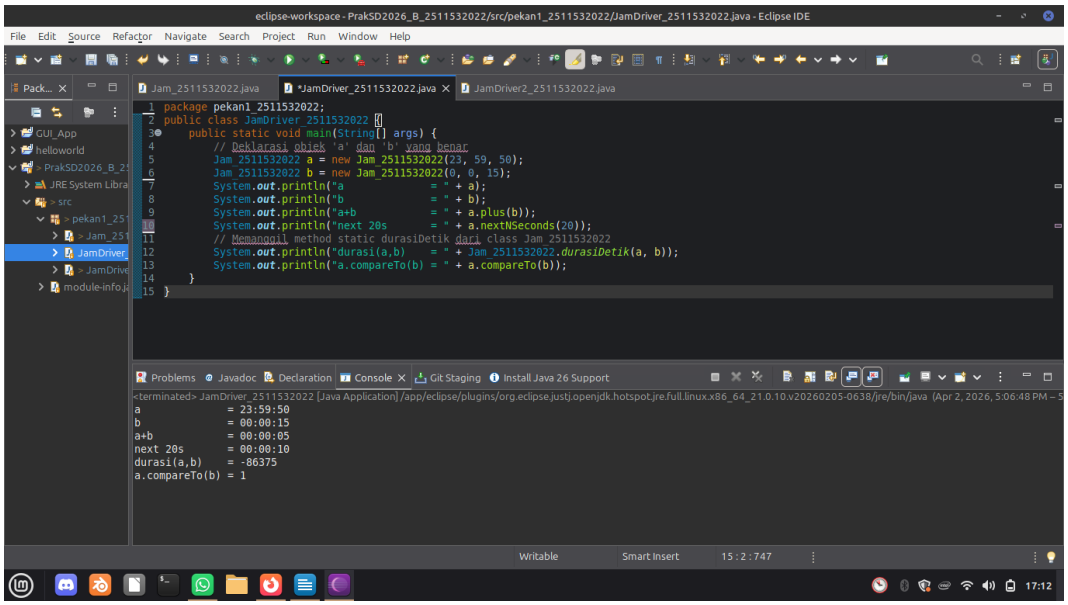
2.2.1 Kode program

```
package pekan1_2511532022;
public class JamDriver_2511532022 {
    public static void main(String[] args) {
        // Deklarasi objek 'a' dan 'b' yang benar
        Jam_2511532022 a = new Jam_2511532022(23, 59, 50);
        Jam_2511532022 b = new Jam_2511532022(0, 0, 15);
        System.out.println("a             = " + a);
        System.out.println("b             = " + b);
        System.out.println("a+b           = " + a.plus(b));
        System.out.println("next 20s      = " + a.nextNSeconds(20));
        // Memanggil method static durasiDetik dari class Jam_2511532022
        System.out.println("durasi(a,b)    = " + Jam_2511532022.durasiDetik(a, b));
        System.out.println("a.compareTo(b) = " + a.compareTo(b));
    }
}
```

2.2.2 Langkah kerja

1. Membuat beberapa objek jam dengan nilai yang sudah ditentukan langsung di dalam kode (*hardcoded*).
2. Memanggil method-method dari kelas `Jam_2511532022` satu per satu.
3. Menampilkan hasil operasi ke layar untuk melihat apakah logika di kelas utama sudah benar.

2.2.3 Analisis hasil



Melalui *driver* ini, diketahui bahwa objek dapat diciptakan dengan sukses. Hasil keluaran menunjukkan bahwa method `toString` dan `toSeconds` bekerja sesuai ekspektasi. Karena tidak ada input dari luar, risiko kesalahan pengguna (user error) tidak ditemukan pada tahap ini.

2.3 Kelas Penguji Interaktif JamDriver2_2511532022

2.3.1 Kode program

```
package pekan1_2511532022;
import java.util.Scanner;
public class JamDriver2_2511532022 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.println("=== Program Driver objek jam ===");
        // 1. Input jam pertama
        System.out.println("\n --- input jam 1 ----");
        Jam_2511532022 j1 = buatJamDariInput(input);
        // 2. Input jam kedua
        System.out.println("\n --- input jam 2 ----"); // Perbaikan teks dari jam 1 ke jam2
```

```
        Jam_2511532022 j2 = buatJamDariInput(input);
        // 3. Menampilkan data
        System.out.println("\n --- hasil operasi ----");
        System.out.println("jam 1 (String)          : " + j1.toString());
        System.out.println("jam 2 (String)          : " + j2.toString());
        // Perbaikan: toSecond() menjadi toSeconds() sesuai class utama
        System.out.println("jam 1 dalam detik       : " + j1.toSeconds());
        System.out.println("jam 2 dalam detik       : " + j2.toSeconds());
        // 4. Operasi Relasional (Perbandingan)
        int perbandingan = j1.compareTo(j2);
        if (perbandingan > 0) {
            System.out.println("Status                  : Jam 1 lebih lambat (setelah) Jam
2"); }
        else if (perbandingan < 0) {
            System.out.println("Status                  : Jam 1 lebih awal (sebelum) Jam
2"); }
        else {
            System.out.println("Status                  : Jam 1 dan Jam 2 sama persis"); }
        // 5. Operasi Aritmatika
        System.out.println("Durasi (J1 ke J2)                    :  "  +
Jam_2511532022.durasiDetik(j1, j2) + " detik");
        Jam_2511532022 jNext = j1.nextSecond();
        System.out.println("Jam 1 Detik Berikutnya   : " + jNext);
        Jam_2511532022 jPrev = j1.prevSecond();
        System.out.println("Jam 1 Detik Sebelumnya   : " + jPrev);
        // 6. Operasi Penjumlahan Jam
        Jam_2511532022 jHasilPlus = j1.plus(j2);
        System.out.println("Hasil J1 + J2           : " + jHasilPlus);
        input.close();
        System.out.println("\nProgram Selesai.");
    }
    /**Prosedur pembantu untuk melakukan input dan validasi secara berulang
```

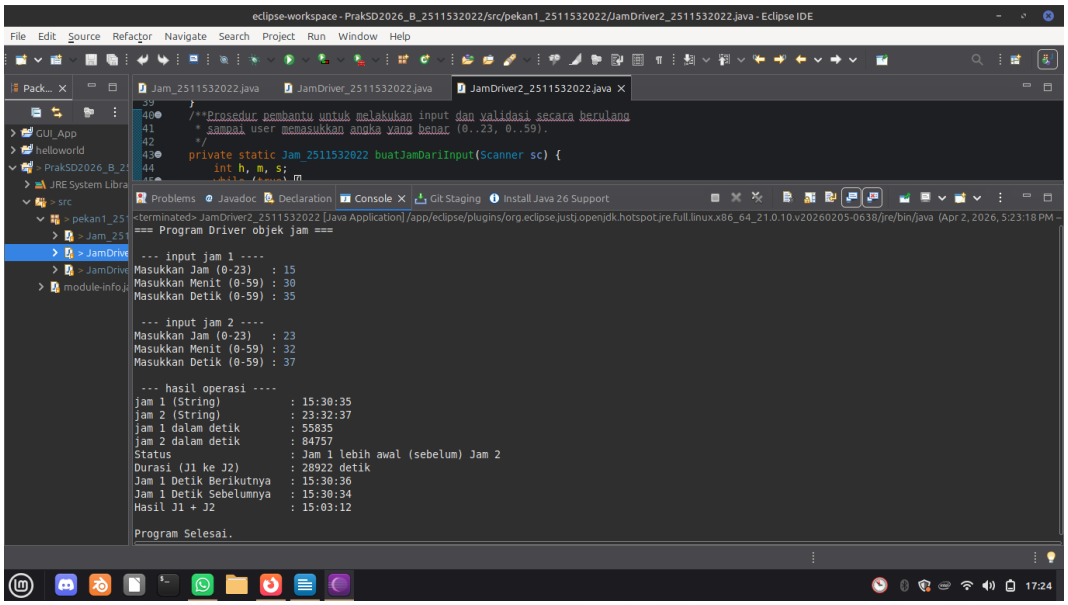
```
     * sampai user memasukkan angka yang benar (0..23, 0..59).
     */
    private static Jam_2511532022 buatJamDariInput(Scanner sc) {
        int h, m, s;
        while (true) {
            System.out.print("Masukkan Jam (0-23)   : ");
            h = sc.nextInt();
            System.out.print("Masukkan Menit (0-59) : ");
            m = sc.nextInt();
            System.out.print("Masukkan Detik (0-59) : ");
            s = sc.nextInt();
            // Memanggil method static isValid dari kelas Jam_2511532022
            if (Jam_2511532022.isValid(h, m, s)) {
                return new Jam_2511532022(h, m, s); } else {
                System.out.println("[Error] Input tidak valid! Silakan ulangi.\n"); }
        }
    }
}
```

2.3.2 Langkah kerja

1) Menginisialisasi objek `Scanner` untuk menangkap input dari keyboard.
2) Membuat method pembantu `buatJamDariInput` yang dilengkapi dengan perulangan `while(true)`.
3) Program meminta input jam, menit, dan detik dari pengguna.
4) Jika input valid, objek jam (`j1` dan `j2`) akan dibuat; jika tidak, pengguna diminta mengulang.
5) Melakukan perbandingan (`compareTo`) dan perhitungan selisih antara dua jam tersebut.

2.3.3 Analisis hasil

Analisis pada *driver* ini menunjukkan tingkat ketahanan program yang lebih baik. Dengan adanya perulangan `while`, program tidak akan berhenti (*crash*) saat pengguna salah memasukkan angka, melainkan memberikan edukasi berupa pesan error. Method `compareTo` juga berhasil menentukan urutan kronologis antara dua waktu yang berbeda secara tepat.



BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan seluruh rangkaian aktivitas praktikum yang telah dilaksanakan pada Pekan 1, dapat ditarik beberapa kesimpulan utama sebagai berikut:

1. Penyusunan kelas `Jam_2511532022` membuktikan bahwa konsep ADT sangat krusial dalam pemrograman. Dengan membungkus atribut jam, menit, dan detik ke dalam satu kesatuan, data menjadi lebih terstruktur dan aman. Hal ini mencegah terjadinya manipulasi data secara ilegal dari luar kelas karena semua akses diatur melalui *method* yang telah ditentukan.
2. Penggunaan *method* `isValid` menunjukkan bahwa sebuah program yang baik tidak hanya harus bisa berjalan, tetapi juga harus mampu menyaring kesalahan manusia (*human error*). Validasi terhadap batasan jam (0–23) serta menit dan detik (0–59) memastikan bahwa objek yang diciptakan selalu merepresentasikan waktu di dunia nyata secara akurat.
3. Ditemukan bahwa melakukan operasi aritmatika (seperti penjumlahan dan pengurangan waktu) jauh lebih efisien dilakukan dalam satuan detik tunggal menggunakan *method* `toSeconds`. Dengan mengubah format jam yang kompleks menjadi satu angka integer total detik, risiko kesalahan logika dalam perhitungan *carry-over* (perpindahan dari 60 detik ke 1 menit) dapat diminimalisir secara signifikan.
4. Implementasi `JamDriver2_2511532022` dengan bantuan kelas `Scanner` dan perulangan `while(true)` memberikan pengalaman pengguna yang lebih baik. Program tidak langsung berhenti (*force close*) saat menerima input yang salah, melainkan mampu memberikan umpan balik kepada pengguna untuk memasukkan data kembali hingga benar. Ini

merupakan prinsip dasar dalam membangun aplikasi yang adaptif dan responsif.

TUGAS PEKAN 1

Kelas Utama

```
package pekan1_2511532022;
public class Mobil_2511532022 {
   // Atribut (Data Mobil)
   private String nama;
   private int tahun;
   private int cc;
   private long harga;
   private String merk;
   // Konstruktor (Untuk membuat objek mobil baru)
   public Mobil_2511532022(String nama, int tahun, int cc, long harga, String merk) {
      this.nama = nama;
      this.tahun = tahun;
      this.cc = cc;
      this.harga = harga;
      this.merk = merk; }
   // --- Selecttor (Getter) ---
   public String getNama() { return nama; }
   public int getTahun() { return tahun; }
   public int getCc() { return cc; }
   public long getHarga() { return harga; }
   public String getMerk() { return merk; }
   // --- Mutator (Setter) ---
   public void setHarga(long hargaBaru) {
      this.harga = hargaBaru; }
   public void setNama(String namaBaru) {
      this.nama = namaBaru; }
   // Method tambahan sesuai tugas
   public void tampilkanInfo() {
      System.out.println("Merk: " + merk + " | Nama: " + nama + " | Tahun: " + tahun + " | CC: " +
cc + " | Harga: Rp" + harga); }
}
```

Kelas Driver:

```
package pekan1_2511532022;
public class MobilDriver_2511532022 {
    public static void main(String[] args) {
        System.out.println("=== Program ADT Mobil - NIM: 2511532022 ===\n");
        // 1. Tambah Mobil (Membuat Objek)
        Mobil_2511532022 mobil1 = new Mobil_2511532022("Regera", 2016, 5000,
90000000000L, "Koenisiegg");
        System.out.println("--- Mobil Berhasil Ditambahkan ---");
        mobil1.tampilkanInfo();
        // 2. --- Setter --- (Mengambil data spesifik)
        System.out.println("\nMengecek CC mobil: " + mobil1.getCc() + " cc");
        // 3. --- Mutator --- (Mengubah harga)
        System.out.println("Ada kenaikan harga...");
        mobil1.setHarga(92000000000L);
        System.out.print("Data terbaru: ");
        mobil1.tampilkanInfo();
        // 4. Hapus Mobil
        mobil1 = null;
        System.out.println("\n--- Mobil Berhasil Dihapus dari Memori ---");
        if (mobil1 == null) {
            System.out.println("Status: Tidak ada data mobil yang tersimpan.");}
    }
}
```